RAG-Based AI Support Assistant – High Level Design (HLD)

1. System Overview

1.1 Problem Definition

Modern organizations handle large volumes of unstructured data such as PDFs, manuals, FAQs, and policy documents. Retrieving accurate information manually is time-consuming and inefficient.

This system solves the problem by:

- Automatically extracting knowledge from documents
- Enabling intelligent question-answering
- Providing fast and context-aware responses using AI

1.2 Scope of the System

The system is designed to:

- Accept user queries via CLI or Web interface
- Process PDF-based knowledge sources
- Retrieve relevant information using vector search
- Generate accurate answers using an LLM
- Route complex queries to humans (HITL)

Out of Scope:

- Real-time internet search
- Multi-language translation (optional future scope)

2. System Architecture

The system consists of the following major components:

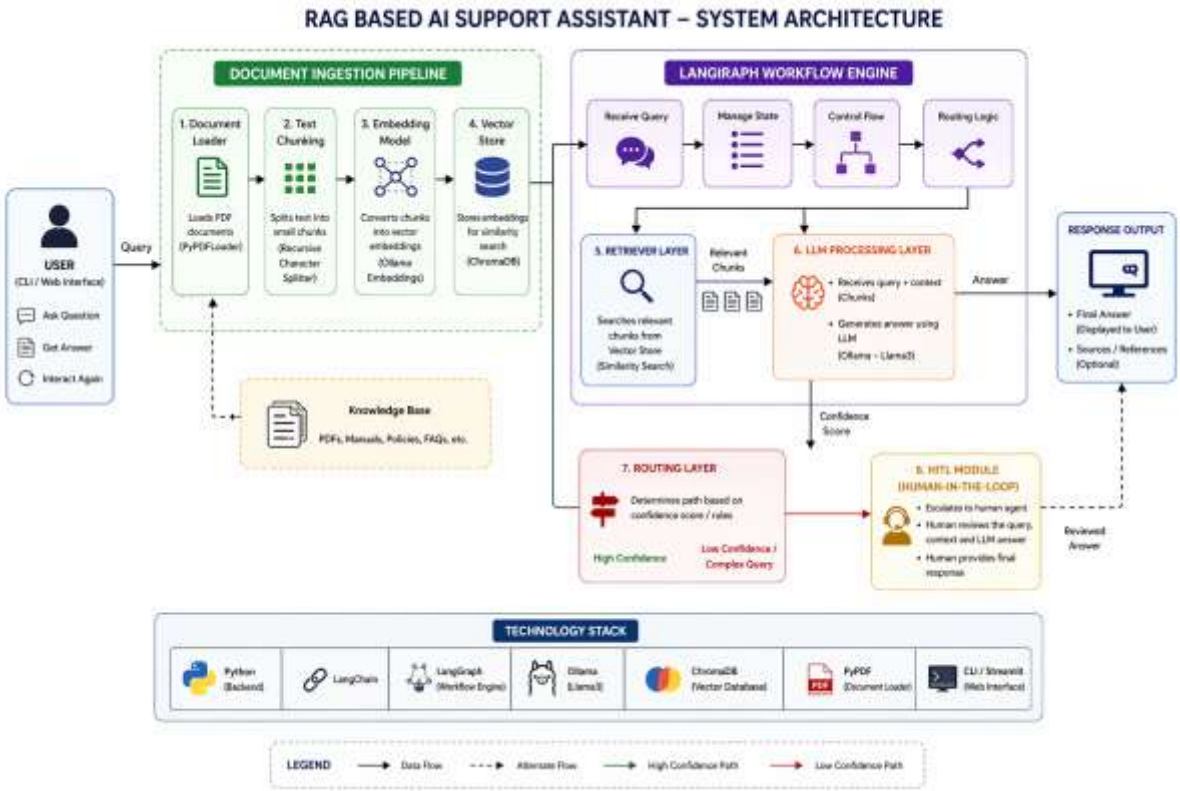
Core Modules

- User Interface (CLI / Web)
- Document Ingestion Pipeline
- Embedding System
- Vector Database
- Retrieval Layer
- LLM Processing Layer
- Workflow Orchestration (LangGraph)

- Human-in-the-Loop (HITL)

Architecture Description

1. User submits a query
2. Query is processed through LangGraph workflow
3. Relevant document chunks are retrieved
4. LLM generates response using context
5. Routing layer decides:
    - Direct response OR
    - Human intervention



3. Component Description

3.1 Document Loader

- Loads PDF documents using PyPDFLoader
- Extracts raw text from documents
- Handles multi-page documents

3.2 Chunking Strategy

- Uses Recursive Character Text Splitter

- Splits text into smaller chunks
- Maintains overlap between chunks
- Improves retrieval accuracy

3.3 Embedding Model

- Converts text into vector embeddings
- Captures semantic meaning of text
- Uses Ollama Embeddings

3.4 Vector Store

- Stores embeddings in ChromaDB
- Enables fast similarity search
- Supports persistent storage

3.5 Retriever

- Retrieves top-K relevant chunks
- Uses similarity search algorithm
- Acts as bridge between query and data

3.6 LLM (Large Language Model)

- Processes query + retrieved context
- Generates natural language response
- Uses Ollama (LLaMA model)

3.7 Graph Workflow Engine (LangGraph)

- Controls overall execution flow
- Maintains system state
- Enables modular pipeline design

3.8 Routing Layer

- Decides next step based on confidence
- Handles:
    - High confidence → Direct answer
    - Low confidence → HITL

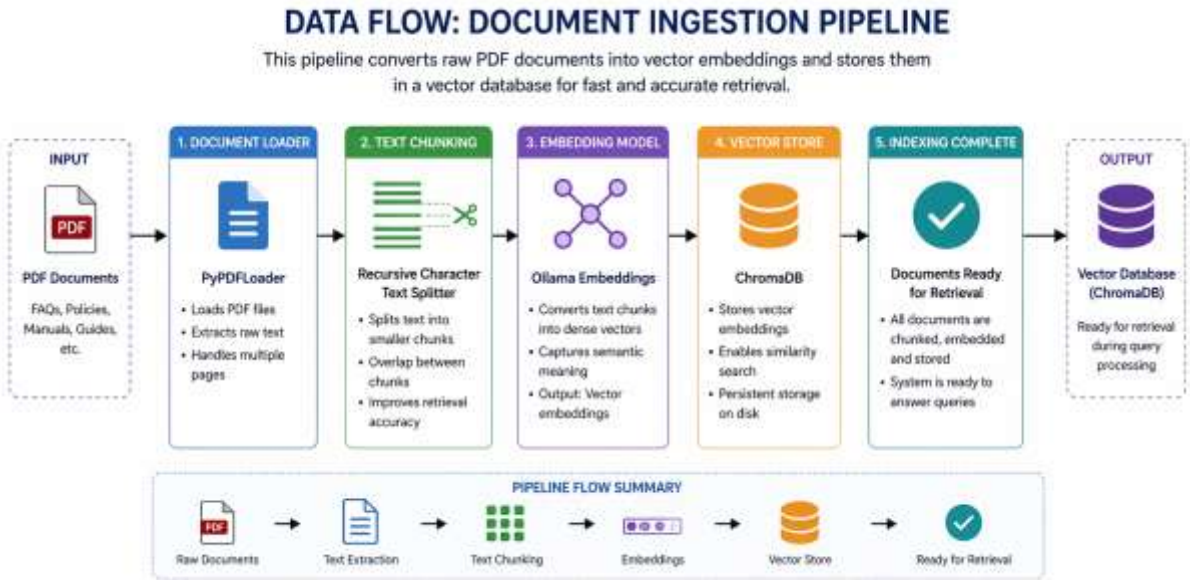
3.9 HITL Module (Human-in-the-Loop)

- Handles complex queries
- Allows human validation
- Improves reliability and trust

4. Data Flow

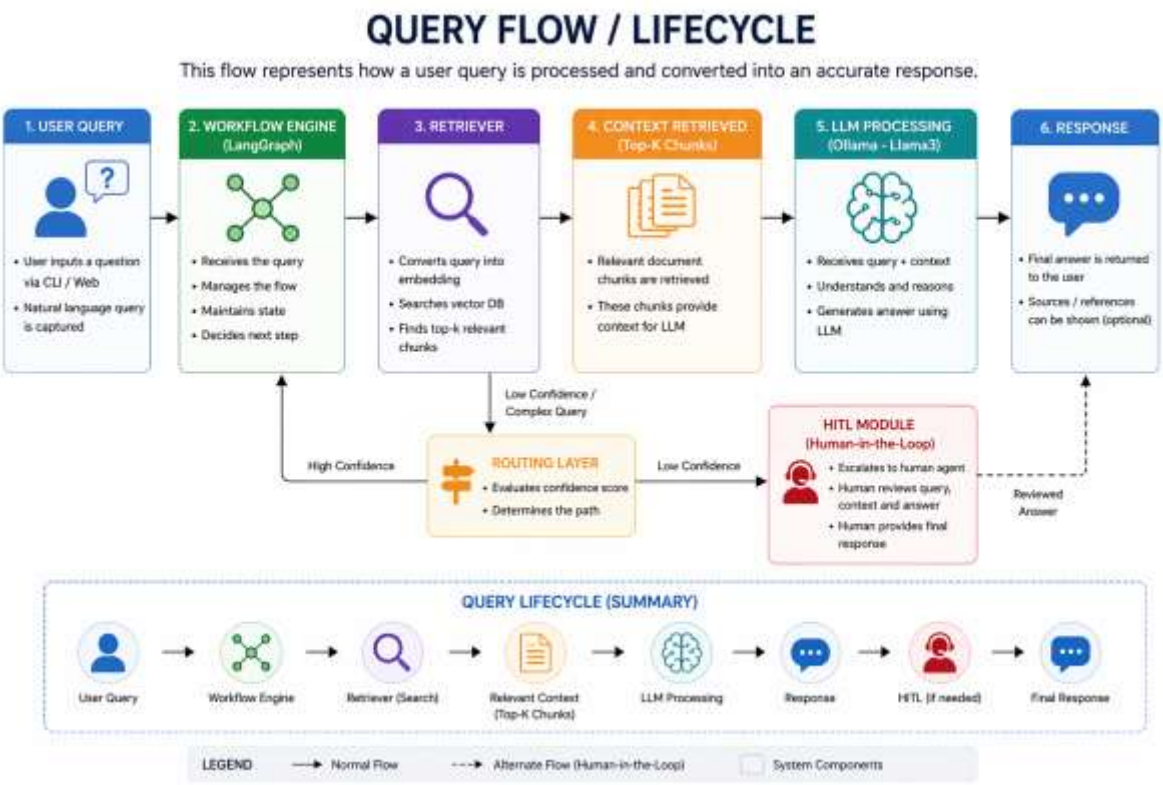
4.1 Document Ingestion Flow

1. PDF documents are loaded
2. Text is extracted
3. Text is split into chunks
4. Chunks are converted into embeddings
5. Stored in vector database



4.2 Query Lifecycle

1. User submits query
2. Query enters LangGraph workflow
3. Retriever fetches relevant chunks
4. Context passed to LLM
5. LLM generates answer
6. Routing layer checks confidence
7. Final response returned OR sent to HITL



5. Technology Choices

5.1 Why ChromaDB

- Lightweight and fast
- Easy integration with LangChain
- Persistent vector storage
- Optimized for semantic search

5.2 Why LangGraph

- Supports workflow orchestration
- Maintains state across steps
- Enables modular and scalable pipelines

5.3 LLM Choice (Ollama – LLaMA)

- Runs locally (privacy-friendly)
- No API cost
- Good performance for RAG tasks

5.4 Additional Tools

- Python → Backend development

- LangChain → RAG pipeline
- PyPDF → Document parsing
- Streamlit (optional) → Web interface

6. Scalability Considerations

6.1 Handling Large Documents

- Efficient chunking strategy
- Incremental document processing
- Use of vector indexing

6.2 Increasing Query Load

- Use caching for frequent queries
- Scale vector DB
- Parallel query processing

6.3 Latency Optimization

- Reduce chunk size intelligently
- Optimize embedding model
- Use faster retrieval techniques

Conclusion

The RAG-based AI Support Assistant provides an efficient, scalable, and intelligent system for document-based question answering. By integrating vector search, LLMs, and workflow orchestration, it ensures accurate and reliable responses while supporting human intervention when necessary.